

CLASE 4 - AMPLIACIÓN DEL TRABAJO PRÁCTICO

Sistema de Gestión Bancaria

Clientes, cuentas, saldos, movimientos y operaciones mediante
Programación Orientada a Objetos

Consigna de trabajo independiente

Este documento describe qué debe lograr el programa, pero no proporciona código, pseudocódigo ni una solución única. Las decisiones de clases, relaciones, nombres, validaciones y algoritmos deberán ser pensadas y justificadas por el estudiante.

Introducción

El proyecto original representa distintos tipos de clientes de un banco. La nueva etapa propone transformar ese modelo informativo en un pequeño sistema bancario capaz de administrar cuentas y ejecutar operaciones reales del dominio, como depositar dinero, retirar fondos, consultar saldos, transferir entre cuentas y conservar un historial de movimientos.

La ampliación no debe consistir solamente en agregar atributos. El objetivo es que los objetos tengan responsabilidades y comportamientos propios, protejan su estado interno y colaboren entre sí. Cada operación deberá respetar reglas que impidan que el sistema llegue a estados inválidos.

Objetivos

- Aplicar herencia, abstracción, encapsulamiento, composición y polimorfismo dentro de un mismo proyecto.
- Diferenciar correctamente un cliente, una cuenta bancaria y un movimiento de dinero.
- Modelar operaciones que modifiquen el estado de una cuenta sin permitir cambios directos e inseguros.
- Utilizar tipos de datos adecuados para dinero, fechas, identificadores y estados.
- Implementar reglas de negocio y comunicar claramente por qué una operación fue aceptada o rechazada.
- Organizar el programa para que pueda ampliarse sin reescribir por completo las clases existentes.

Duración orientativa

El alcance obligatorio está pensado para unas cinco horas de trabajo concentrado, suponiendo que el proyecto original ya compila. Al final se incluyen desafíos opcionales para convertirlo en una entrega aún más completa.

1. Punto de partida y revisión del modelo

Antes de agregar cuentas y operaciones, se deberá revisar el proyecto original. La ampliación parte de la misma jerarquía de clientes, pero exige corregir nombres, tipos de datos y responsabilidades que puedan generar confusión.

Requisitos iniciales

- Configurar Maven para utilizar Java 21 y confirmar que el proyecto pueda abrirse en otro equipo sin depender de una configuración exclusiva del IDE.
- Mantener una clase base abstracta para representar las características compartidas por todos los clientes.
- Conservar, como mínimo, clientes persona física, clientes empresa y clientes premium o preferenciales.
- Corregir los nombres del dominio: una empresa se identifica mediante CUIT; los nombres de clases, atributos y métodos deben ser claros y consistentes.
- Utilizar String para documentos, teléfonos y números que funcionan como identificadores, ya que no se realizan operaciones matemáticas con ellos.
- Utilizar BigDecimal para toda cantidad monetaria.

Desafío de modelado obligatorio: el cliente premium

Se deberá decidir qué significa ser premium dentro del sistema. Puede considerarse un tipo concreto de cliente, una especialización de una persona física o una categoría de servicio que podría aplicarse a diferentes clientes. La consigna no impone una única respuesta, pero la elección debe evitar duplicación innecesaria y quedar justificada brevemente en el README.

Principio central

Un cliente no es lo mismo que una cuenta. El cliente representa a la persona o empresa que mantiene una relación con el banco. El saldo y las operaciones pertenecen a una cuenta bancaria. Un cliente podrá ser titular de una o más cuentas.

Preguntas que deben resolverse antes de programar

- ¿Qué información es realmente común a todos los clientes y debe vivir en la clase base?
- ¿Qué datos identifican a una persona y cuáles identifican a una empresa?
- ¿Debe poder modificarse el identificador de un cliente después de crearlo?
- ¿Ser premium describe la identidad del cliente o el nivel de servicio que recibe?
- ¿Qué clase deberá conocer las cuentas de un cliente?

2. Nueva entidad principal: la cuenta bancaria

Se deberá incorporar una representación de cuenta bancaria. Esta entidad será responsable de conservar el saldo y controlar las operaciones que lo modifican. No se aceptará almacenar un único saldo directamente dentro del cliente, porque un mismo cliente puede poseer varias cuentas con saldos diferentes.

Información mínima de toda cuenta

- Número o identificador único de cuenta.
- Cliente titular.
- Saldo actual.
- Moneda de la cuenta.
- Fecha de apertura.
- Estado operativo, por ejemplo activa, bloqueada o cerrada.
- Colección de movimientos realizados.

Jerarquía de cuentas

El sistema deberá modelar al menos dos tipos de cuentas que compartan las operaciones generales, pero tengan reglas diferentes. Se proponen las siguientes:

- Caja de ahorro: no permite retirar ni transferir más dinero que el saldo disponible.
- Cuenta corriente: puede permitir un saldo negativo hasta un límite de descubierto previamente establecido.

El estudiante deberá decidir qué información y comportamiento pertenecen a la clase base de cuenta y cuáles corresponden exclusivamente a cada subtipo. Las diferencias no deberán resolverse preguntando repetidamente de qué clase es cada objeto desde el programa principal.

Encapsulamiento obligatorio

El saldo no podrá tener un setter público ni modificarse directamente desde Main, Banco o Cliente. Solo las operaciones válidas de la propia cuenta podrán aumentarlo o disminuirlo.

Decisiones a justificar

- ¿La cuenta debe comenzar siempre con saldo cero o puede recibir un saldo inicial?
- ¿Quién genera el número único de cuenta y cómo se evita que se repita?
- ¿Qué comportamiento debe compartir toda cuenta y cuál debe variar según su tipo?
- ¿Una cuenta cerrada debe permanecer en el sistema y conservar su historial?
- ¿Qué información de la cuenta debería ser inmutable luego de la apertura?

3. Operaciones obligatorias sobre una cuenta

Cada cuenta deberá ofrecer operaciones públicas que expresen acciones del dominio bancario. Estas operaciones tendrán que validar la solicitud antes de modificar el saldo y deberán informar claramente cuando no puedan completarse.

Depositar dinero

- Recibir un monto y sumarlo al saldo de la cuenta.
- Rechazar montos nulos, iguales a cero o negativos.
- Rechazar la operación si el estado de la cuenta no permite operar.
- Registrar un movimiento de depósito solamente cuando la operación se complete.

Retirar dinero

- Recibir un monto y descontarlo del saldo.
- Aplicar las validaciones generales de monto y estado.
- Respetar la regla particular del tipo de cuenta: una caja de ahorro no puede quedar en negativo; una cuenta corriente no puede superar su descubierto autorizado.
- No alterar el saldo ni registrar movimientos cuando el retiro sea rechazado.

Consultar saldo y dinero disponible

- Permitir consultar el saldo contable actual sin modificarlo.
- Distinguir, si corresponde, el saldo actual del dinero total disponible para retirar. En una cuenta corriente, el disponible puede incluir el descubierto autorizado.

Cambiar el estado de la cuenta

- Permitir bloquear una cuenta para impedir nuevas operaciones.
- Permitir reactivarla cuando las reglas definidas lo autoricen.
- Definir qué condiciones deben cumplirse antes de cerrar una cuenta.

Regla de consistencia

Una operación fallida no debe producir cambios parciales. Si un retiro es rechazado, el saldo y el historial deben conservar exactamente el estado que tenían antes del intento.

4. Transferencias entre cuentas

El sistema deberá permitir transferir dinero desde una cuenta de origen hacia una cuenta de destino. Esta operación combina una salida y una entrada de dinero, pero debe percibirse como una única acción bancaria.

Validaciones mínimas

- Las cuentas de origen y destino deben existir y poder operar.
- El monto debe ser positivo.
- La cuenta de origen y la cuenta de destino no pueden ser la misma.
- La cuenta de origen debe disponer del dinero requerido según las reglas de su tipo.
- Las monedas deben ser compatibles. En esta primera versión, una transferencia entre monedas distintas deberá rechazarse en lugar de inventar un tipo de cambio.

Resultado esperado

- El saldo de la cuenta de origen disminuye exactamente por el monto transferido.
- El saldo de la cuenta de destino aumenta por el mismo monto.
- Ambas cuentas registran el movimiento correspondiente.
- Los dos movimientos deben poder relacionarse con una misma operación de transferencia.
- Si alguna validación falla, ninguna de las dos cuentas debe cambiar.

Preguntas de razonamiento

- ¿Qué validaciones deben realizarse antes de tocar cualquiera de los dos saldos?
- ¿Qué objeto debería coordinar la transferencia: la cuenta de origen, el banco o un servicio especializado?
- ¿Cómo se puede demostrar que el débito y el crédito pertenecen a la misma transferencia?
- ¿Qué ocurriría si se modifica primero una cuenta y luego falla la modificación de la otra?

No se busca una arquitectura empresarial

La solución puede funcionar completamente en memoria. No se requiere base de datos, interfaz gráfica, Spring Boot ni conexión a servicios externos. La prioridad es que el modelo orientado a objetos sea coherente y verificable.

5. Historial de movimientos

Toda operación monetaria completada deberá generar un movimiento. El historial permitirá explicar cómo se llegó al saldo actual y servirá como una pequeña auditoría de la cuenta.

Información mínima de un movimiento

- Identificador único.
- Tipo de movimiento: depósito, retiro, transferencia enviada o transferencia recibida.
- Monto.
- Fecha y hora exactas.
- Descripción o referencia comprensible.
- Saldo resultante luego de completar la operación.
- Identificador común de operación cuando forme parte de una transferencia.

Reglas del historial

- Un movimiento representa un hecho ocurrido y no debería poder modificarse libremente después de registrarse.
- Los intentos rechazados no deben aparecer como movimientos exitosos.
- El historial debe exponerse sin permitir que otra clase elimine o agregue elementos directamente a la colección interna.
- Los movimientos deben conservar el orden cronológico en el que fueron realizados.

Consulta de extracto

La cuenta deberá poder mostrar un extracto legible que incluya sus datos principales, el saldo actual y todos sus movimientos. Como ampliación opcional, se podrá consultar por rango de fechas o por tipo de movimiento.

Inmutabilidad como decisión de diseño

No todos los atributos necesitan setter. Cuando un dato representa un hecho histórico, permitir su modificación puede romper la confiabilidad del sistema. Se deberá decidir qué campos nacen completos y permanecen inmutables.

Preguntas de razonamiento

- ¿Por qué guardar el saldo resultante en cada movimiento puede facilitar una auditoría?
- ¿Cómo se evita que el código externo borre un movimiento de la lista?
- ¿Debe existir un movimiento si una operación fue rechazada y, si no es así, cómo se informará ese rechazo?

6. Administración mediante una clase Banco

Se deberá incorporar un objeto que represente al banco o al sistema bancario. Su responsabilidad será coordinar el registro y la búsqueda de clientes y cuentas, sin apropiarse de las reglas internas que pertenecen a cada cuenta.

Responsabilidades mínimas

- Registrar clientes y rechazar documentos repetidos.
- Abrir cuentas y evitar números de cuenta duplicados.
- Asociar cada cuenta con un cliente existente.
- Buscar clientes por identificador o documento.
- Buscar cuentas por número o identificador.
- Listar todos los clientes registrados.
- Listar todas las cuentas o las cuentas pertenecientes a un cliente determinado.
- Coordinar transferencias si esa fue la responsabilidad elegida por el estudiante.

Demostración obligatoria de polimorfismo

Los diferentes tipos de clientes deberán almacenarse y recorrerse mediante referencias de la clase base Cliente. Del mismo modo, los diferentes tipos de cuentas deberán poder tratarse como cuentas generales. Al solicitar una operación sobrescrita, el programa deberá ejecutar el comportamiento correspondiente al objeto real.

No se considerará una buena demostración de polimorfismo resolver todas las diferencias mediante extensas cadenas de comprobaciones de tipo en Main. Las decisiones que dependen del subtipo deberán ubicarse, cuando sea razonable, dentro de la jerarquía correspondiente.

Distribución de responsabilidades

Main debe construir un escenario y demostrarlo; no debe contener todas las reglas del negocio. Banco administra colecciones y búsquedas. Cuenta protege el saldo y ejecuta operaciones. Movimiento describe un hecho. Cliente representa al titular.

Preguntas de razonamiento

- ¿Qué clase debe impedir que se registre dos veces el mismo documento?
- ¿Qué clase sabe si una cuenta puede quedar en negativo?
- ¿Qué información necesita el banco para abrir una cuenta correctamente?
- ¿Cómo demuestra el programa que puede trabajar con distintos subtipos sin conocerlos de antemano?

7. Reglas de negocio y manejo de errores

El programa deberá protegerse de entradas inválidas. No alcanza con imprimir un mensaje después de haber dejado un objeto en un estado incorrecto. Las validaciones deben realizarse antes de aplicar cambios.

Reglas obligatorias

- No aceptar montos monetarios nulos, iguales a cero o negativos en depósitos, retiros y transferencias.
- No aceptar ingresos declarados, límites de crédito o descubiertos negativos.
- No aceptar nombres, documentos, direcciones o correos obligatorios vacíos.
- No aceptar una fecha de nacimiento futura.
- No permitir operaciones monetarias sobre cuentas bloqueadas o cerradas.
- No permitir que una caja de ahorro quede con saldo negativo.
- No permitir que una cuenta corriente exceda su límite de descubierto.
- No permitir clientes con el mismo documento ni cuentas con el mismo número.
- No permitir transferencias entre cuentas incompatibles por moneda en esta versión.

Comunicación de errores

El estudiante deberá elegir una estrategia consistente para comunicar operaciones inválidas. Podrá utilizar excepciones estándar apropiadas o crear excepciones propias para situaciones importantes, como saldo insuficiente o cuenta inactiva. La decisión deberá permitir que Main demuestre el error sin finalizar abruptamente todo el programa.

Casos que deben probarse manualmente

- Depósito válido y depósito con monto negativo.
- Retiro válido y retiro superior al disponible.
- Uso permitido y uso excedido del descubierto de una cuenta corriente.
- Transferencia válida y transferencia rechazada.
- Operación intentada sobre una cuenta bloqueada.
- Registro de un documento o número de cuenta duplicado.

Uso correcto de BigDecimal

Todas las cantidades de dinero deben utilizar BigDecimal. El estudiante deberá investigar cómo comparar montos y evitar depender de comparaciones propias de tipos primitivos.

8. Programa principal y demostración requerida

El programa principal deberá construir un escenario reproducible que permita al profesor observar todas las funcionalidades sin tener que editar el código. No se exige un menú interactivo; una demostración ordenada y claramente rotulada es suficiente.

Escenario mínimo

- 1 Crear al menos una persona física, una empresa y un cliente premium o preferencial.
- 2 Registrar los tres clientes en el banco.
- 3 Abrir al menos una caja de ahorro y una cuenta corriente, vinculadas con sus respectivos titulares.
- 4 Realizar varios depósitos válidos.
- 5 Realizar un retiro válido y otro que deba ser rechazado.
- 6 Demostrar el comportamiento particular del descubierto de la cuenta corriente.
- 7 Realizar una transferencia válida entre cuentas compatibles.
- 8 Intentar una transferencia inválida y comprobar que ningún saldo haya cambiado.
- 9 Bloquear una cuenta e intentar operar con ella.
- 10 Mostrar el saldo final y el historial completo de cada cuenta.
- 11 Recorrer clientes y cuentas mediante sus tipos base para demostrar polimorfismo.

Presentación por consola

- Separar visualmente cada parte del escenario.
- Indicar qué operación se está intentando y si se espera que sea aceptada o rechazada.
- Mostrar saldos antes y después de las operaciones importantes.
- Evitar que la salida dependa de explicaciones externas para poder entenderse.

Entregables

- Proyecto Maven configurado para Java 21.
- Código fuente organizado por paquetes y con nombres consistentes.
- README con objetivo, instrucciones de ejecución, decisiones de modelado y funcionalidades implementadas.
- Salida de demostración funcional desde Main.
- Opcional de alto valor: pruebas automatizadas de las reglas principales.

9. Plan de trabajo sugerido - cinco horas

El orden propuesto reduce el riesgo de construir muchas piezas antes de confirmar que el modelo central funciona. Los tiempos son orientativos y pueden variar.

Etapa	Trabajo esperado	Tiempo
1. Revisión	Corregir el proyecto original, configurar Java 21 y definir la relación entre cliente y premium.	45 min
2. Cuentas	Crear la abstracción de cuenta, sus dos tipos y la asociación con el titular.	75 min
3. Operaciones	Implementar depósito, retiro, consulta de disponible y estados con sus validaciones.	75 min
4. Movimientos y banco	Registrar historial, administrar colecciones, búsquedas y transferencia entre cuentas.	60 min
5. Demostración	Completar Main, probar errores, mejorar salida, revisar nombres y documentar decisiones.	45 min

Criterio para detenerse

Si el tiempo es limitado, priorizar operaciones correctas, validaciones y polimorfismo. Un menú, una interfaz visual o una gran cantidad de atributos no compensan un saldo que puede modificarse de forma insegura.

10. Criterios de evaluación

Criterio	Qué se observará	Valor
Modelado	Separación coherente entre cliente, cuenta, banco y movimiento; asociaciones correctas.	20%
Encapsulamiento	Saldo protegido, colecciones no manipulables desde afuera y objetos siempre válidos.	20%
Herencia y polimorfismo	Jerarquías útiles, sobrescritura con sentido y uso de referencias de tipos base.	20%
Reglas de negocio	Depósitos, retiros, descubiertos, estados y transferencias consistentes.	20%
Funcionamiento	Escenario completo, errores controlados y resultados verificables por consola.	10%
Calidad	Java 21, tipos adecuados, nombres claros, paquetes, README y ausencia de duplicación innecesaria.	10%

11. Desafíos opcionales para una entrega sobresaliente

Estos desafíos no forman parte del núcleo de cinco horas. Conviene elegir uno o dos y terminarlos correctamente en lugar de comenzar muchos y dejarlos incompletos.

Nivel recomendado - alto impacto

- Pruebas automatizadas: verificar depósitos, fondos insuficientes, descubierto, transferencia atómica y cuenta bloqueada.
- Extracto filtrado: consultar movimientos por rango de fechas o por tipo.
- Límite diario de transferencias: acumular lo transferido durante el día y rechazar excesos.
- Beneficio premium: definir una ventaja concreta y coherente, como un límite diario superior o una bonificación, y demostrarla mediante polimorfismo.

Nivel avanzado

- Aplicación de intereses a la caja de ahorro utilizando una tasa definida y registrando el resultado como movimiento.
- Cobro de comisiones según el tipo de cuenta, evitando valores negativos o cobros duplicados.
- Cuentas con más de un titular, revisando cómo cambia la relación entre cliente y cuenta.
- Registro separado de intentos rechazados para auditoría, sin confundirlos con movimientos monetarios exitosos.
- Persistencia simple en archivos, solamente después de que el modelo en memoria funcione y esté probado.

Qué suele impresionar más

Una demostración breve donde una misma operación produce comportamientos correctos en diferentes tipos de cuenta, conserva el historial y rechaza casos inválidos suele demostrar más dominio de POO que una interfaz grande con reglas incompletas.

12. Lista final de comprobación

- El proyecto compila y se ejecuta con Java 21.
- Cliente y cuenta son conceptos separados.
- Cada cuenta posee un titular, saldo, moneda, estado e historial.
- Existen al menos dos tipos de cuenta con reglas diferentes.
- El saldo solo cambia mediante operaciones controladas.
- Depósitos, retiros y transferencias validan antes de modificar objetos.
- Una operación rechazada no cambia saldos ni crea movimientos exitosos.
- Las transferencias actualizan ambas cuentas de manera consistente.
- Banco administra clientes y cuentas sin duplicados.
- Main demuestra casos válidos e inválidos y utiliza polimorfismo.
- Los nombres son claros, CUIT y DNI están correctamente modelados y el dinero usa BigDecimal.
- El README explica las decisiones importantes sin limitarse a describir archivos.

Meta del trabajo

Al terminar, el programa no deberá ser solamente una colección de datos. Deberá representar un pequeño dominio bancario en el que cada objeto protege su información, cumple una responsabilidad clara y colabora con otros objetos para ejecutar operaciones seguras.